Limit Order Book and Matching Engine: Project Report

C++20 portfolio project, Phases 0 to 8. Report dated 21 October 2026.

Executive summary

This is a C++20 engine that matches limit orders by price-time priority, and it was verified more heavily than it was optimised. A fresh checkout builds from the documented commands and passes all 336 tests.

- **What it does.** Matches orders for one or many instruments, replays event files deterministically, takes concurrent input on worker threads, and recovers its books from a write-ahead log after a crash.
- **Why it can be trusted.** An independent checker, random property tests, sanitizers and mutation testing (deliberately injected bugs). All 22 bugs injected in Phases 7 and 8 were caught.
- **What was measured.** Between 1.9 and 9.4 million commands per second on a shared one-CPU cloud VM, depending on workload. A pooled allocator made four of five workloads 22 to 27 percent faster. Logging costs about 2x without fsync, and recovery runs at about 2.2 million commands per second.
- **What it does not show.** Any parallel speedup, because the machine had one CPU. The GitHub CI workflow has also never run, and only gcc was available locally.

1. Summary

This project is a deterministic limit order book and matching engine written in C++20. It accepts orders, keeps a two-sided book, matches trades by price-time priority, replays event files, and is verified by tests and measured by benchmarks. It follows a phased roadmap (Phases 0 to 6) for an exchange-simulation portfolio project. I then added two phases of my own: Phase 7 (multiple instruments and a concurrent pipeline) and Phase 8 (a write-ahead log with crash recovery).

The finished project has about 9,900 lines of C++ (library, tools, tests and benchmarks), 336 automated tests, and about 20,000 words of documentation. A fresh checkout builds from the documented commands and all 336 tests pass.

Headline results:

- **Correctness.** Every behaviour is specified, tested against an independent checker, and attacked with mutation testing (injected bugs). Truncating the write-ahead log at every byte always recovers exactly a prefix of the commands.
- **Performance.** On a shared one-CPU cloud VM the baseline engine ran between 1.9 and 9.4 million commands per second depending on workload. One measured optimisation (a pooled allocator) made four of five workloads 1.22x to 1.27x faster, and gave no reliable gain on the fifth.
- **Concurrency and durability.** Threaded output is byte-identical to sequential output. Logging costs about 2x without fsync. No parallel speedup is claimed, because the machine had one CPU.

The guiding rule throughout was to claim only what was measured, so each section says what the evidence does not show as well as what it does.

2. Purpose, scope and principles

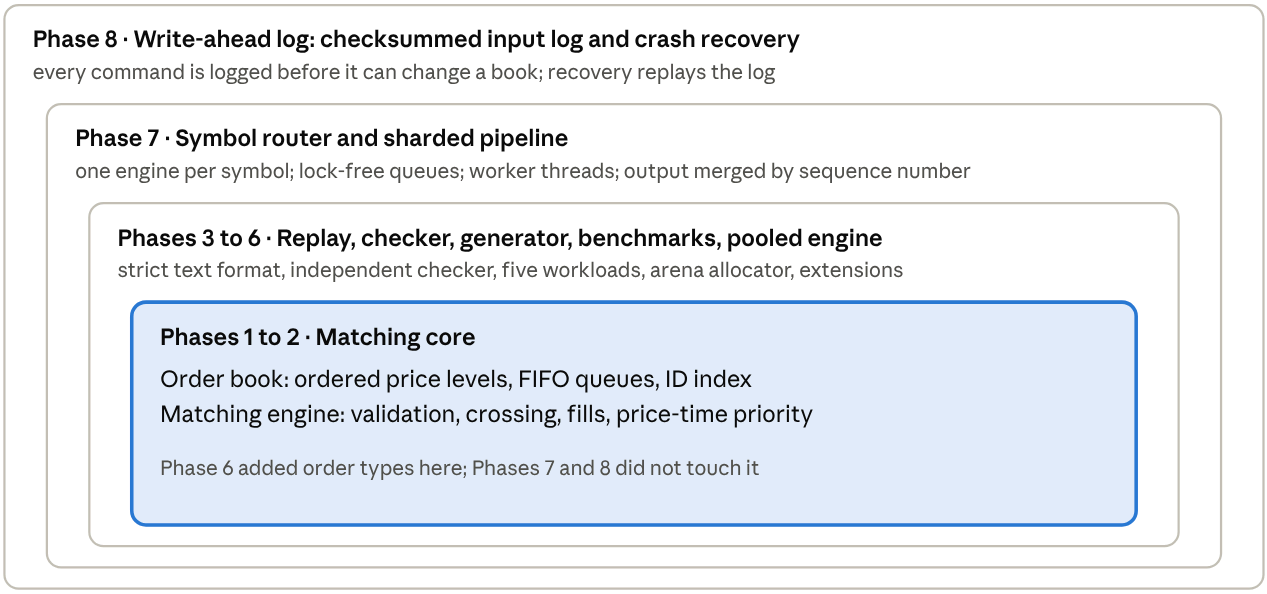
The roadmap asks for a project that demonstrates exchange mechanics, data structures, correctness and performance engineering. It is an independent portfolio project, not an endorsed specification. Four principles shaped every decision:

1. **Correctness first, then speed.** Matching rules were written down before any code, and no container was replaced until profiling justified it.
2. **Determinism.** The same input must always produce the same output, on both engine implementations and, later, on any number of threads. Determinism is what makes replay, testing and recovery possible.
3. **Measure, do not assume.** Every performance statement comes with the machine, compiler, flags and method, and negative or inconclusive results are reported.
4. **Layering.** The matching core stayed single-threaded and unchanged from Phase 2 onward. Instruments, threads and persistence were added around it, so earlier guarantees could not be broken by later work.

3. Architecture and structure

The design separates what the rules are (the engine) from how it is exercised (tools) and how it is checked (tests and benchmarks). The matching code has no networking, file I/O or user interface, which keeps it testable in isolation.

Phases 7 and 8 wrap the matching core without changing it



project layers · Phases 1 to 8

Each outer layer is tested against the one inside it: threaded output must equal sequential output, and recovered books must equal the uninterrupted run.

Component	Responsibility
Order, types, events	Order fields, enums, and the auditable events: Accepted, Rejected, Trade, Cancelled, Expired, Amended
PriceLevel, OrderBook	FIFO queue at one price; sorted bid and ask levels; ID index; snapshots
MatchingEngine	Validation, crossing, fills, resting, cancel, amend
Replay, parser, generator, checker	Strict text format, deterministic runner, seeded workload generator, independent consistency checker
Pooled engine (Phase 5)	Same engine on a custom arena allocator
Multi-instrument layer (Phase 7)	Symbol table, router, sharded pipeline, lock-free queue
Write-ahead log (Phase 8)	Checksummed input log, scanner, recovery

Component	Responsibility
Benchmarks	Five workloads, statistics, environment capture, scaling and logging benchmarks

The repository follows the suggested layout, extended for later phases:

Folder	Contents	Approx. lines
include/lob	Public types and interfaces	1,568
src	Book, engine, replay, pipeline, log	2,163
apps	lob_replay, lob_gen, lob_shell, lob_mreplay, lob_mgen, lob_recover	501
tests	Unit, property, golden, regression, CLI tests	4,163
benchmarks	Workloads, statistics, scaling and log benchmarks	1,529
docs, data/sample, scripts, .github	Design notes, results, samples, helper scripts, CI	n/a

Tests make up over 40 percent of the code, which reflects how much weight the project puts on verification.

4. The matching engine (Phases 0 to 2)

Data model and why

Prices are 64-bit integer ticks and quantities are unsigned 64-bit integers. Floating point is never used as a key, so comparisons are exact. Valid prices are limited to the range 1 to half of the maximum integer, which guarantees that the spread (ask minus bid) can never overflow. Quantities are unsigned, so remaining quantity cannot go negative, and the book rejects any insertion that would overflow a level total.

Data structures and why

Structure	Choice	Cost
Bids and asks	Ordered map from price to level (descending for bids)	Best price O(1); add or remove a level O(log L)
Level queue	Linked list of orders plus cached total	Append O(1); erase by handle O(1)
ID index	Hash map from order ID to a locator	Lookup O(1) average

The locator holds the side, price and a list iterator. Linked-list iterators stay valid while other elements change, so the index can point straight at an order and cancellation is a lookup plus an O(1) erase. These are deliberately simple baselines. They are cache-unfriendly, and Phase 4 later showed exactly how much that costs.

Matching rules

The rules are written in full in the matching specification. The essentials:

- A buy crosses an ask when the ask price is at or below the buy limit; equal prices cross.
- Best price executes first; orders at the same price execute first-in first-out.
- Execution always happens at the resting order's price, so an aggressive taker gets price improvement.
- A partially filled resting order keeps its queue position. A partially filled taker's remainder rests at the back of its level.
- Validation happens in a fixed order, and any failure produces exactly one Rejected event with the book untouched. Duplicate IDs are rejected only while the original order is live. Cancelling an unknown ID is rejected.
- One Trade event is emitted per resting order touched, and executed quantity can never exceed what is available.

Phase 6 added market orders, time-in-force (GTC, IOC, FOK) and amend. FOK is atomic: availability is computed first, so a killed order leaves the book exactly as it was. An amend that only reduces quantity at the same price keeps queue priority; any other change loses priority and is processed as an atomic replacement.

5. Verification

Verification is layered, so no single weak test can hide a defect.

Layer	Purpose
Unit tests	Each rule in isolation (empty book, one-sided, multi-level, FIFO, partial fills, cancel)
Typed tests	The same tests run against both engine implementations
Property tests	Seeded random order and cancel streams checked after every command by an independent checker and a naive model
Golden and regression files	Exact expected output for sample files; every discovered failure saved as a permanent case
CLI tests	Whole-program determinism and exit codes
Sanitizers	AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer builds
Mutation testing	Deliberately break the code and confirm tests fail

The checker recomputes invariants independently of the engine: sorted levels, unique live IDs, no crossed resting book, non-negative quantities, and conservation of quantity (accepted plus amended-in equals twice the traded quantity plus cancelled, expired, resting and amended-out).

Mutation testing was the most valuable layer. It found real problems that ordinary tests missed. The project keeps a defect log of 15 entries, found by its own verification. Examples: the order book was implicitly copyable and a copy aliased the original's list nodes (found by review, reproduced under ASan); the checker shared a helper with the engine, so a market-order bug was invisible until a mutant survived; and a benchmark memory metric hid allocator overhead, making two engines look identical.

Test counts by phase: 254 tests through Phase 6, 53 more in Phase 7, and 29 more in Phase 8, for 336 in total.

6. Performance measurement (Phase 4)

Method

Five workloads were designed to stress different behaviour: non-crossing (every order rests), cancel-heavy, crossing-heavy (tight price band), many-levels (20,000 levels per side), and bursty (quiet updates then multi-level sweeps). Each run uses a monotonic clock, 20,000 warm-up commands, 5 repetitions of 200,000 measured commands, a fixed seed, and a measured clock overhead of 23 ns. Output is machine-readable CSV, and the machine, compiler and flags are recorded. The benchmark refuses to run in Debug or sanitizer builds.

Results

Machine: shared single-CPU Intel Xeon 2.10 GHz virtual machine, Linux 6.18, gcc 13.3, Release build. No CPU pinning.

Workload	Ops/s (median)	p50 ns	p95 ns	p99 ns	p99.9 ns	Allocs/op
non_crossing	9,374,274	101	166	289	2,510	2.00
cancel_heavy	6,707,230	154	319	409	2,301	1.00
crossing_heavy	8,032,087	127	322	464	2,749	1.39
many_levels	1,902,577	548	866	1,071	8,613	1.04
bursty	8,387,976	123	388	740	3,066	1.27

What the evidence supports

Run-to-run spread within a workload was 11 to 31 percent, so differences below roughly 30 percent between workloads are not distinguishable on this machine. Three findings stand well outside that noise:

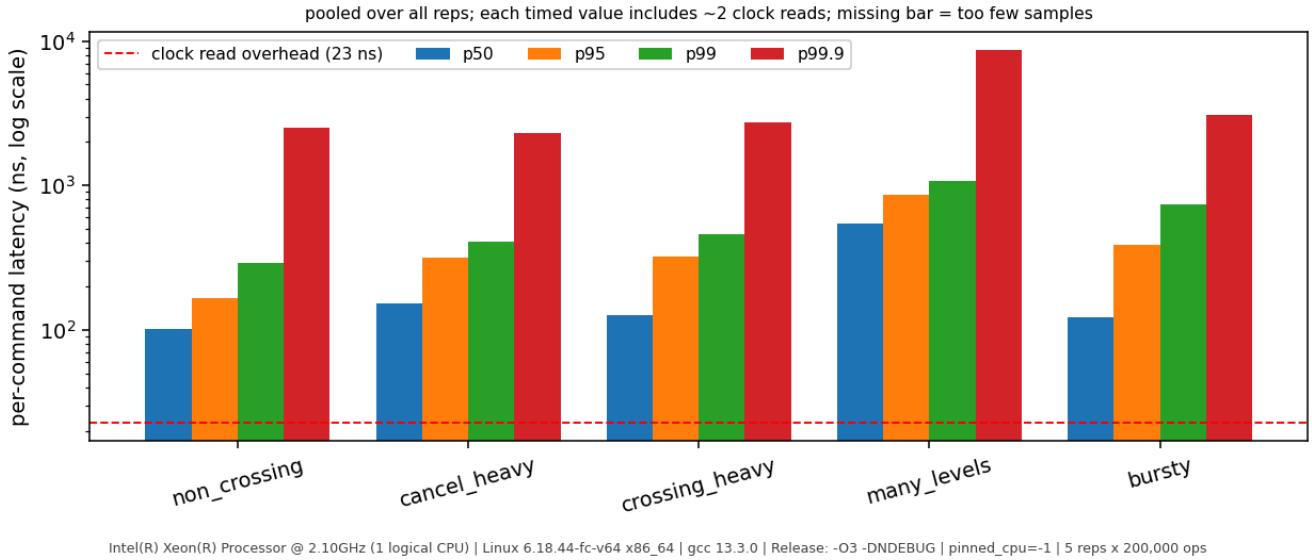
1. **many_levels is the outlier**, about 4 to 5 times slower per command at the median. It does not execute more instructions (582 per op against 846 for non_crossing). A simulated cache model shows far more misses, which is consistent with memory-latency-bound pointer chasing through map, list and hash nodes. This was not confirmed with hardware counters, because perf is unavailable in the sandbox, and the report says so.

2. **Allocation is the largest block of work everywhere.** Instruction profiling attributes 30 to 51 percent of executed instructions to the allocator. An insert costs two allocations and a cancel frees two.
3. **The hash index causes deterministic latency spikes** while the book grows, which is why p99.9 is much larger than p99.

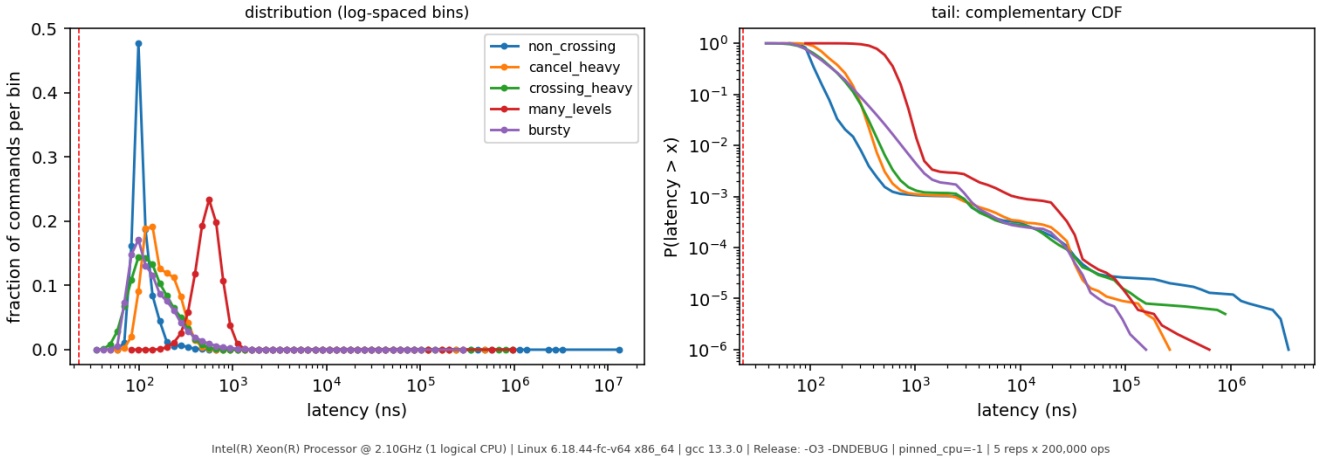
The maximum latencies (up to 12 ms) are dominated by virtual machine interference and are not reported as findings.

The figures below are the project's own benchmark output for Phase 4. The first shows the four percentiles for each workload on a log scale, with the measured clock overhead as a dashed line. The second shows the full latency distribution and its tail.

Latency percentiles by workload



Latency distribution by workload (red dashed = clock read overhead)

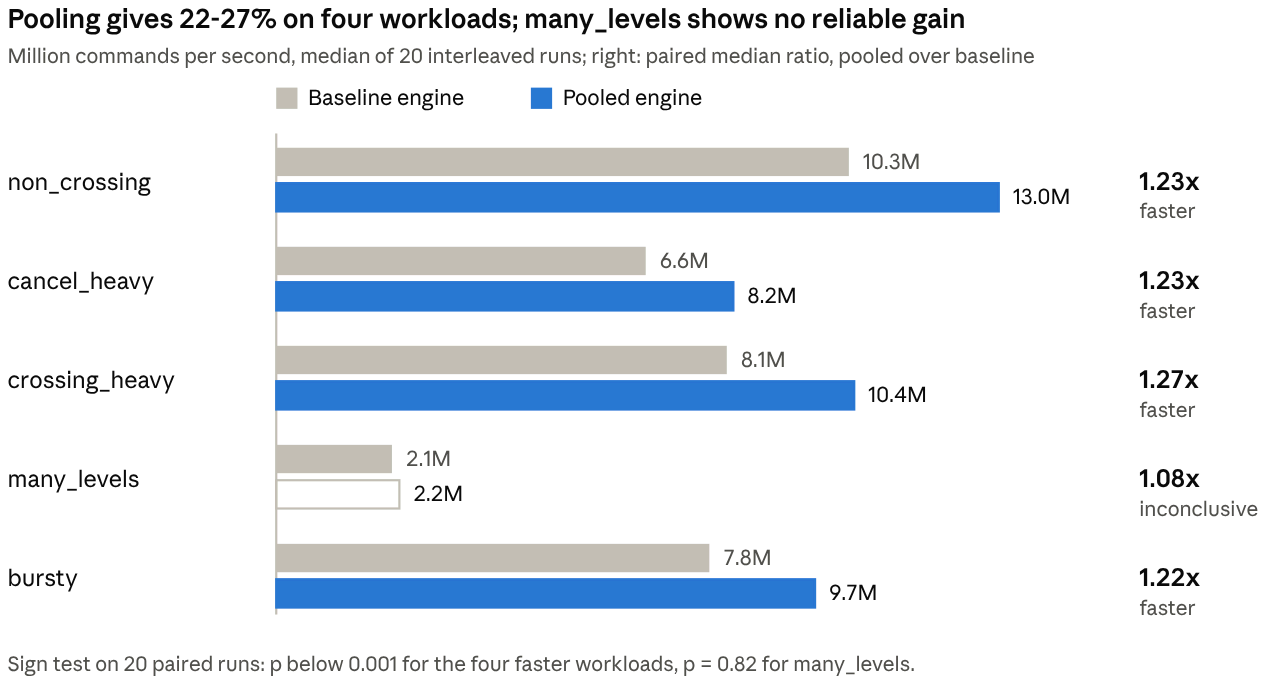


In the distribution plot the many_levels curve sits to the right of the other four, with its peak near 550 ns against roughly 100 to 150 ns, which is the cache-miss cost described above. In the tail plot every workload has a long tail reaching hundreds of microseconds, consistent with the hash rehash spikes and with virtual machine interference.

7. The measured optimisation (Phase 5)

Profiling pointed at allocation, so the optimisation targeted it: an arena-based pooled engine with the same interface and the same behaviour. A baseline mode was kept so the comparison is fair, and both engines run the identical correctness suite.

The comparison used 20 repetitions in which the two engines alternate and swap who goes first, so machine drift cannot favour either. The statistic is the ratio within each repetition, and a workload counts as faster only if a sign test gives p below 0.01 (the 0.05 threshold divided by five workloads).



results_phase5.md · 20 interleaved repetitions x 200,000 commands per workload · one-CPU cloud VM

The mechanism is confirmed: allocations per command fell from between 1 and 2 to about zero, and instructions per command fell 31 to 46 percent. Yet throughput rose only 22 to 27 percent, so the remaining time is not instruction-bound. Tail latency improved too; for example non_crossing p99.9 fell from 1.5 microseconds to 0.4.

Two honest negatives matter. First, many_levels executes 31 percent fewer instructions but gained nothing reliable, because its cost is cache misses that pooling does not remove. Second, I had speculated that allocating a list node and a hash node together would improve cache locality; the data did not support that, and the report states it. The trade-off is added complexity: a custom allocator that must be kept correct, which is why both engines share one test suite.

8. Extensions (Phase 6)

The roadmap lists optional extensions, and all of them were built, one at a time, each with explicit semantics and tests: market orders, time-in-force rules (GTC, IOC, FOK), amend with a defined priority rule, a terminal view (lob_shell) and CSV event output. Documentation covers the order lifecycle, complexity and known limitations.

Mutation testing of this phase found two real problems. The most instructive was that the checker had shared a helper function with the engine, so a bug in market-order handling was invisible to both. The checker was rewritten to compute the limit independently, and that mutant is now killed by four separate layers. A generator bug that produced non-canonical market-order prices was caught by a text round-trip test.

9. Multiple instruments and concurrency (Phase 7)

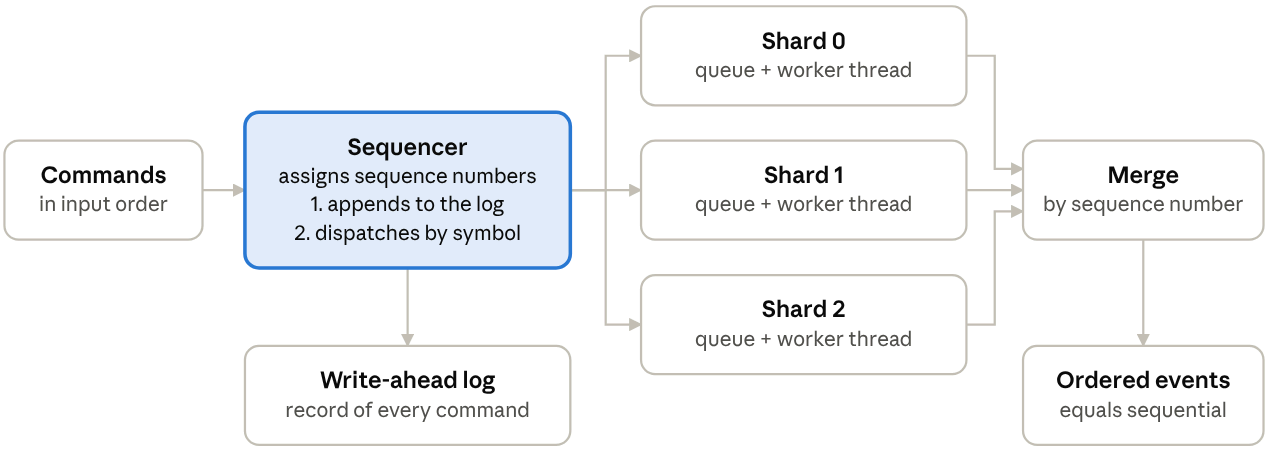
The roadmap's last interview question asks what would change for multiple instruments or concurrent input. Phase 7 answers it by building both without touching the matching core.

Design

- **One independent engine per symbol.** Order IDs are scoped per symbol, so books share no state and can live on different threads. A cancel must name its symbol. Venue-wide IDs were considered and rejected because they need a global ID index that every thread must consult.
- **Text format.** Each line is prefixed with the symbol, for example at-sign AAPL then a normal command. A line without a prefix is an error, so a forgotten prefix can never route an order to the wrong book.
- **Sharded pipeline.** One sequencer thread assigns a global sequence number and routes each command by symbol. Each shard has a lock-free single-producer single-consumer queue and one worker thread that owns its engines. After the workers

finish, outputs are merged by sequence number.

The sequencer logs each command first, then routes it to a shard by symbol



pipeline and write-ahead log · 3 shards shown

Workers share no state, so merging their output by sequence number gives the same stream as the sequential run.

Why the output is deterministic

A symbol lives on exactly one shard and queues are first-in first-out, so each book sees its commands in input order. Workers share no mutable state, and merging by sequence number restores the global order. The merged output is therefore identical to the sequential run for any shard count and any thread timing.

Verification

The sequential run is the oracle. Differential tests compare the pipeline with it for 1 to 16 shards on both engines, with tiny queues that force backpressure, skewed load, and repeated runs. Each symbol is also checked against the original single-book engine. ThreadSanitizer reported no races. Ten injected bugs were applied: nine were caught by ordinary tests. The tenth, a queue that published its tail index with relaxed instead of release ordering, passes every normal build on x86 and was caught only by ThreadSanitizer. That is why the continuous integration workflow has a ThreadSanitizer job.

Results

The machine had one CPU, so parallel speedup was physically impossible and none is claimed. What was measured on 1,000,000 commands over 64 symbols (every run gated on matching the sequential statistics):

Mode	Commands/s	Versus sequential
Sequential, counting only	5.22 M	1.00x
1 shard	4.26 M	0.82x
2 shards	4.80 M	0.92x
4 shards	4.73 M	0.91x

This is the overhead the machinery costs when no parallelism is available: about 8 to 18 percent, and 9 to 26 percent with ordered output. The more transferable finding is the partition bound, computed from the workload alone. Uniform load over 64 symbols splits almost perfectly, but with 80 percent of the flow on one symbol, no shard count can exceed about 1.2x, because one book is the unit of parallelism.

10. Durability (Phase 8)

Phase 8 makes state recoverable. Every command is appended to a checksummed log before it can change a book, and after a crash the books are rebuilt by replaying the log.

Why log inputs

The engine is a deterministic function of its command sequence, so the ordered commands are a complete description of every book at about 40 bytes per command. Recovery reuses the engine that is already trusted, so there is no second code path that could disagree with live matching.

Design

- **Write-ahead ordering.** The log is written first, then the command is applied. If the write fails, the command is not applied. In the pipeline, the sequencer writes the log, so a threaded run and a sequential run produce byte-identical logs.
- **Record format.** Fixed-size records, each with a CRC-32 checksum and no length field that could itself be corrupted, plus a strict sequence number and dense symbol ids.

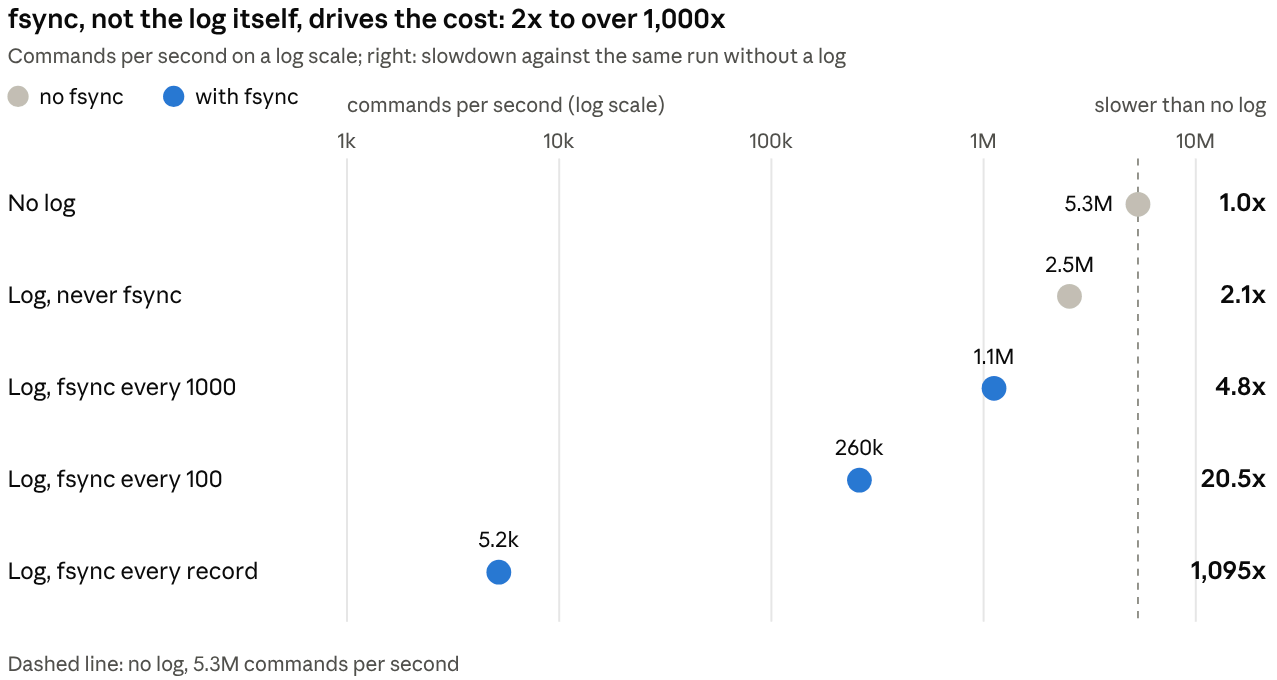
- **Torn tail versus corruption.** A crash mid-write damages only the last record, which is discarded safely. Bad data followed by more data is reported as corruption and never silently repaired, because dropping it would lose acknowledged commands. Damage within the final 44 bytes cannot be told apart from a torn write, and the documentation states this limit plainly.
- **Failure poisons the writer**, so a half-written record can never be followed by more data.

Verification

The log is tested by property: truncating a real log at every byte offset (about 2,300 cuts) always recovers exactly the state after some prefix of the commands, and flipping every bit is detected unless it falls in the final record. A seeded fuzzer of 4,000 damaged logs runs cleanly under AddressSanitizer. A test using a full device confirms a failed write means the command was not applied. Twelve injected bugs were all caught, although one only after I noticed by reasoning that the test for writer poisoning was too weak and strengthened it.

Results

Measured on the same VM, with the log on an ext4 virtual disk:



results_phase8.md · lob_wal, 500,000 commands (20,000 for fsync every record), 5 repetitions · ext4 on a virtio disk, one-CPU VM

Recovery ran at about 2.2 million commands per second. Writing the log costs roughly 200 ns per command; I did not profile where it goes, so the report lists candidates rather than naming a cause. The fsync cost (about 190 microseconds here) belongs to this virtual disk and does not transfer to other storage. The standard remedy, group commit, is not built.

11. Limitations and future work

The limitations are documented, not hidden:

- **No measured parallel speedup**, because only a one-CPU machine was available. The scaling benchmark is built and needs a multi-core machine.
- **Noisy benchmarks.** All numbers come from a shared virtual machine; they are relative baselines, not performance claims. Hardware-counter explanations were not confirmed because perf is unavailable.
- **Continuous integration has never run on GitHub**, and only gcc was available locally, so the clang half of the matrix is untested.
- **Recovery replays the whole log**, with no checkpoints or rotation. Threaded output is merged after the fact and skips mid-stream snapshot commands. Shard assignment is static, so a hot symbol caps speedup. No self-trade prevention, no real network input, no replication.

The most valuable next steps are running the scaling benchmark on a multi-core machine, adding checkpoints and group commit, streaming output with a sequence watermark, and profiling the logging overhead.

12. Conclusion

The project delivers a small, well-tested engine with credible measurements, which is the roadmap's stated aim. Three ideas carry most of its value. Determinism makes replay, testing and recovery possible. Differential and mutation testing prove the tests can fail, not only that they pass. And reporting inconclusive results (many_levels, the missing speedup, the unprofiled logging cost) keeps every claim defensible. A reviewer can build it, run it, read the specification, and check each number against recorded raw data.

Glossary

Term	Meaning
Price-time priority	Better prices execute first; at one price, the order that arrived first executes first
Resting order	An order sitting in the book waiting for a counterparty
Taker and maker	The incoming order that trades (taker) and the resting order it trades against (maker)
Tick	The smallest price step; prices are stored as whole numbers of ticks
IOC and FOK	Immediate-or-cancel fills what it can and discards the rest; fill-or-kill fills completely or not at all
Deterministic	The same input always gives the same output, whatever the thread timing
Property test	A test that checks a rule over many random inputs instead of fixed examples
Mutation testing	Deliberately injecting a bug to confirm the tests fail; a bug nobody notices is a test gap
Equivalent mutant	A code change that does not alter behaviour, so no test can or should catch it
Sanitizer	A compiler tool that catches memory errors (ASan), undefined behaviour (UBSan) or data races (TSan) while tests run
p99.9	The latency that 99.9 percent of commands beat; one in a thousand is slower
Pooled (arena) allocator	Hands out memory from large pre-allocated blocks instead of calling the heap for each order
Sequencer	The single thread that gives each command its order number and routes it to a shard
Shard	A group of symbols handled by one worker thread; each symbol lives on exactly one shard

Term	Meaning
SPSC queue	A lock-free queue with exactly one writer thread and one reader thread
Partition bound	The best speedup a symbol split allows: 1 divided by the busiest shard's share of commands
Write-ahead log	A file in which each command is recorded before it is applied, so state can be rebuilt after a crash
Torn write	A record cut short or garbled by a crash during writing
CRC-32	A checksum that detects damage to a record
fsync	An operating system call that forces buffered file data onto storage
Group commit	Sharing one fsync across many commands (not built)